UK FINANCIAL RULES AI PLATFORM • MASTER ARCHITECTURE & REFERENCE

Full System Architecture, Folder Structure & Function Execution Guide

Complete technical specification covering the entire repository structure, the separate execution flows for both the Normal AI Chat (/chat) and Embeddable Widget (/test & /widget), and an exhaustive function-by-function reference table.

Stack: React 18 + Vite (Frontend :8080) • Express 5 (Backend :5000) • Qdrant • MongoDB • Groq LLM

Sourcebooks: FCA (PRIN, SYSC, FIT, DISP, PROD, FCG, SUP, COBS, CASS) + PRA + BoE **Generated:** September 2026

01 Complete Master Folder Structure (All Folders & Files)

Below is the complete directory structure for the entire application. Files specifically powering the **Normal AI Chat** are marked in **orange**, and files powering the **Embeddable AI Widget** are marked in **green**:

```
AI-Agent-UK-Finance/
├── Frontend/ # React 18 + Vite Client Application (Port 8080)
│   ├── public/
│   │   └── widget.js # Standalone embed script loader (injects floating launcher + iframe)
│   ├── src/
│   │   ├── api/
│   │   │   ├── client.js # Axios instance: attaches JWT Bearer token & auto-unwraps response.data
│   │   │   ├── agent.api.js # Bridge for /chat & Widget: queryAgent(), conversations, widget sessions
│   │   │   ├── auth.api.js # Login, registration, token refresh, and user administration
│   │   │   └── regulatory.api.js # Regulatory document upload, chunk inspection, and deletion
│   │   ├── components/
│   │   │   ├── Navbar.jsx # Global header (auto-hidden on /widget and /test routes)
│   │   │   ├── CitationDrawer.jsx # Slide-out drawer displaying verbatim FCA rule text & relevance score
│   │   │   ├── NotificationBell.jsx # Real-time compliance alerts & ingestion updates
│   │   │   ├── ProtectedRoute.jsx # Route guard enforcing authentication & ADMIN role
│   │   │   ├── UkFinanceLogo.jsx # Official UK Financial Rules platform branding SVG
│   │   │   └── widget/
│   │   │       └── ComplianceWidget.jsx # Core Widget: floating launcher, dimmed backdrop, scroll lock, compact chat
│   │   ├── context/
│   │   │   └── AuthContext.jsx # React Context: auth state, JWT token storage, user roles
│   │   ├── pages/
│   │   │   ├── Home.jsx # Public marketing & feature overview landing page
│   │   │   ├── Login.jsx # Dual-mode authentication (Sign In & Registration with Show/Hide eye)
│   │   │   ├── employee/ChatAssistant.jsx # FULL-SCREEN NORMAL AI CHAT: sidebar history, search, rename, delete, citations
│   │   │   ├── WidgetChat.jsx # /widget route: Studio & Embed Code Generator + <iframe> container
│   │   │   ├── TestExternalWebsite.jsx # /test route: Simulated 3rd-party bank website with integrated floating widget
│   │   │   └── admin/
│   │   │       ├── AdminDashboard.jsx # Ingestion stats, active vector counts, user metrics & Widget launcher
│   │   │       ├── UploadDocument.jsx # PDF/DOCX regulatory document upload with AI Gatekeeper verification
│   │   │       ├── RegulatoryRules.jsx # Live vector knowledge base explorer & chunk viewer
│   │   │       ├── AdminUsers.jsx # Team & role management with password visibility toggle
│   │   │       └── CostAnalysis.jsx # Gross token analytics, cache hit rates & USD cost tracking
│   │   ├── App.jsx # Master React Router configuration
│   │   ├── main.jsx # React DOM root with ChakraProvider & QueryClientProvider
│   │   └── theme.js # Enterprise financial design system tokens
│   ├── index.html # HTML entry point
│   ├── package.json # Frontend dependencies
│   └── vite.config.js # Vite configuration with proxy to port 5000
└── Backend/ # Node.js + Express REST API Server (Port 5000)
    ├── uploads/ # Temp binary PDF/DOCX storage during document parsing
    ├── src/
    │   ├── config/
    │   │   ├── env.js # Environment variable validation via Zod
    │   │   └── db.js # MongoDB connection lifecycle manager
    │   ├── middlewares/
    │   │   ├── auth.middleware.js # authenticate, optionalAuthenticate, authorizeRoles('ADMIN')
    │   │   ├── error.middleware.js # Centralized error handler
    │   │   └── rateLimiter.middleware.js # Express rate limiting middleware
    │   ├── models/
    │   │   ├── conversation.model.js # Mongoose schema: userId, widgetSessionId, source: 'WIDGET'|'WEB_APP', messages
    │   │   ├── queryHistory.model.js # Historical compliance queries, tokens, cache hits & costs
    │   │   ├── regulatoryDocument.model.js # Indexed sourcebooks: title, authority, chunkCount, tokenCount
    │   │   ├── user.model.js # User schema: email, password hash, role (ADMIN/EMPLOYEE)
    │   │   ├── promptCache.model.js # MongoDB L2 exact cache collection
    │   │   └── notification.model.js # System notifications
    │   ├── routes/
    │   │   ├── agent.routes.js # POST /query, GET/DELETE /widget/session/:id, conversations CRUD, cost-analysis
    │   │   ├── auth.routes.js # /register, /login, /refresh-token, /me, /users
    │   │   ├── regulatoryDocument.routes.js # /upload, /all, /:id, /chunks
    │   │   └── notification.routes.js # /all, /:id/read
    │   ├── controllers/
    │   │   ├── agent.controller.js # askAgent, getWidgetSession, clearWidgetSession, conversations CRUD
    │   │   ├── auth.controller.js # Auth logic & user management
    │   │   ├── regulatoryDocument.controller.js # Upload, AI Gatekeeper verification & vector indexing
    │   │   └── notification.controller.js # Notification management
    │   ├── services/
    │   │   ├── ragAgent.service.js # Core RAG engine: pronoun contextualization, 4-tier cache, Groq LLM synthesis
    │   │   ├── promptCache.service.js # 4-Tier Cache Manager: L1 RAM, L2 MongoDB, L3 Qdrant Semantic Cache
    │   │   ├── vectorStore.service.js # Qdrant search & multi-sourcebook round-robin interleaving
    │   │   ├── qdrant.service.js # Low-level Qdrant client connection & collection lifecycle
    │   │   ├── aiProvider.service.js # Groq SDK client & Xenova ONNX embedding model
    │   │   ├── fcaParser.service.js # Structure-aware chunking for FCA Handbook
    │   │   └── documentParser.service.js # PDF & Word document text extraction
    │   ├── app.js # Express application setup, Helmet security & CORS
    │   └── server.js # HTTP server bootstrap on port 5000
    ├── package.json # Backend dependencies
    └── .env # Port, MongoDB URI, Qdrant URL, Groq API Key
```

02 Normal AI Chat Flow (Full-Screen Assistant at /chat)

The **Normal AI Chat** is located at `Frontend/src/pages/employee/ChatAssistant.jsx`. It provides a comprehensive enterprise copilot experience with a multi-conversation sidebar, real-time citation inspector drawer, detailed Markdown tables, and token usage analytics.

Phase 1: Chat Mount & Conversation Thread Loading

`ChatAssistant.jsx • agent.api.js`

- On mount, `loadConversations()` executes: `agentApi.listConversations()` → `GET /api/agent/conversations`.
- Backend returns the user's past 60 conversation threads sorted by pinned status and last updated timestamp.
- When a user clicks a past thread in the sidebar, `handleSelectConversation(convId)` calls `agentApi.getConversation(convId)` → `GET /api/agent/conversations/:id`, populating `messages` state with past turns.

Phase 2: Conversation Management (New, Rename, Delete)

`ChatAssistant.jsx • agent.controller.js`

- **New Chat:** `handleNewChat()` resets `activeConversationId = null` and displays the welcome card.
- **Rename Thread:** `handleSaveTitle(convId)` calls `agentApi.updateConversation(convId, { title })` → `PATCH /api/agent/conversations/:id`.
- **Delete Thread:** `handleDeleteConversation(convId)` calls `agentApi.deleteConversation(convId)` → `DELETE /api/agent/conversations/:id`.

Phase 3: User Submits Question → Contextual Memory Preparation

`ChatAssistant.jsx • agent.api.js`

- `handleSend()` appends `{ role: 'user', content: query }` to local state.
- Maps prior turns into `historyTurns` (including previous assistant `citedRules`).
- Dispatches: `agentApi.queryAgent(query, activeConversationId, historyTurns)` → `POST /api/agent/query`.
- Axios client (`client.js`) automatically attaches `Authorization: Bearer <token>`.

Phase 4: Backend Controller Routing & Thread Creation

`agent.controller.js • conversation.model.js`

- `askAgent(req, res)` validates `question.trim().length ≥ 2`.
- If `activeConversationId` exists, finds `Conversation.findOne({ _id: conversationId, userId })`.
- If fresh chat, creates `Conversation.create({ userId, title: generateTitleFromQuestion(q), messages: [] })`.
- Delegates to RAG: `processEmployeeQuery(trimmedQuestion, req.user, { conversationHistory, isWidget: false })`.

Phase 5: Full Comprehensive RAG Synthesis with Markdown Tables

`ragAgent.service.js • Groq LLM`

- Because `isWidget` is `false`, Rule 6 instructs Groq LLM: *"When comparing rules or displaying multiple requirements, present the key distinctions in a clean Markdown table."*
- Evaluates 4-tier cache → on miss, searches Qdrant for top 5 rule clauses → synthesizes comprehensive Markdown table with verbatim FCA excerpts.
- Calculates gross prompt/completion tokens and estimated USD cost.

Phase 6: Response Rendering & Interactive Citation Drawer

`ChatAssistant.jsx • CitationDrawer.jsx`

- Renders full response with Markdown tables, bold terms, and statutory citation badges.
- Clicking any citation tag opens `<CitationDrawer />`, displaying the verbatim rule text, FCA authority, and relevance score.
- Updates sidebar conversation title and timestamp in real time.

03 AI Compliance Widget Flow (/test & /widget)

The **AI Compliance Widget** is powered by `Frontend/src/components/widget/ComplianceWidget.jsx`. It provides a lightweight, focused compliance interface optimized for embedding inside third-party websites without clutter.

Phase 1: Independent Session Management

`ComplianceWidget.jsx • localStorage`

- `getOrCreateWidgetSessionId()` generates a dedicated `wgt_<timestamp>_<random>` ID stored in `localStorage`.
- `useEffect` calls `agentApi.getWidgetSession(widgetSessionId)` → `GET /api/agent/widget/session/:widgetSessionId`.
- Backend restores past messages from MongoDB (`Conversation.findOne({ widgetSessionId })`) without requiring main app login.

Phase 2: Screen Dimming & Background Scroll Lock

`ComplianceWidget.jsx`

- When the widget is opened (`isOpen === true`), `document.body.style.overflow = 'hidden'` prevents the host page from scrolling.
- A fixed backdrop overlay (`bg="rgba(12, 22, 20, 0.58)"` + `backdropFilter="blur(3.5px)"`) dims the host website.
- Clicking the dimmed background closes the widget and restores host scrolling.

Phase 3: Widget Query Submission with isWidget Flag

`agent.api.js • agent.controller.js`

- `handleSend()` dispatches: `agentApi.queryAgent(questionText, conversationId, priorHistory, widgetSessionId)`.
- `askAgent` in `agent.controller.js` identifies `cleanWidgetSessionId` and creates/updates a `source: 'WIDGET'` thread.
- Passes `isWidget: true` to `processEmployeeQuery()`.

Phase 4: Multi-Turn Pronoun Resolution Across Turns

`ragAgent.service.js`

- `isConversationalFollowUp()` detects relative references (*"they"*, *"them"*, *"their"*, *"it"*, *"that period"*).
- `buildContextualSearchQuery()` extracts active sourcebook codes from previous turns and synthesizes a contextualized search query.

Phase 5: WIDGET CONCISE MODE & Intelligent Table Suppression

`ragAgent.service.js • formatConciseWidgetAnswer`

- Checks `askedForTable = /\b(table|tabular|columns)\b/i.test(question)`.
- **If user did NOT ask for a table:** Instructs Groq LLM to return a concise 3–5 bullet-point summary (max 150 words) with bold rule citations (`**DISP 1.6.2 R**`). Strictly forbids Markdown tables.
- Even on cache hits from the main chat, `formatConciseWidgetAnswer()` and `formatConciseWidgetMessage()` automatically convert wide tables into clean bullet points.
- **If user explicitly asks for a table:** Generates the full Markdown table as requested.

Phase 6: Clean Clutter-Free Message Rendering

`ComplianceWidget.jsx`

- Removed bottom citation dropdown bar (`5 Citations v`) to keep message bubbles compact and readable.
- Automatically renders 2 clickable follow-up chips and auto-scrolls to the latest answer.

04 Master Function-by-Function Reference Table

The table below provides a complete reference for every primary function across both Frontend and Backend, detailing its source file, internal logic, and the next function/API it invokes:

File Name & Path	Function Name	What It Does Inside	What It Calls Next
`Frontend/src/components/widget/ComplianceWidget.jsx`	`getOrCreateWidgetSessionId()`	Checks `localStorage` for existing widget session ID; generates `wgt_<time>_<rnd>` if absent and saves it.	Returns `widgetSessionId` string
`Frontend/src/components/widget/ComplianceWidget.jsx`	`restoreSession()` (in useEffect)	Initializes session on mount; calls API to fetch prior messages stored in MongoDB for this widget session.	Calls `agentApi.getWidgetSession(` `setMessages()`
`Frontend/src/components/widget/ComplianceWidget.jsx`	`handleSend(customQuestion)`	Captures input text, appends user message to UI state, formats prior history array, sets loading state, and calls API.	Calls `agentApi.queryAgent()` , the `formatConciseWidgetMessage()`
`Frontend/src/components/widget/ComplianceWidget.jsx`	`formatConciseWidgetMessage(text, query)`	Client-side safety parser: converts Markdown tables into bullet points if user did not ask for a table; removes trailing disclaimers.	Returns sanitized concise markdown `ReactMarkdown`
`Frontend/src/components/widget/ComplianceWidget.jsx`	`handleResetSession()`	Calls API to delete current widget conversation from MongoDB, creates new session ID in `localStorage` , and clears state.	Calls `agentApi.clearWidgetSessio` updates state
`Frontend/src/pages/employee/ChatAssistant.jsx`	`loadConversations()`	Fetches the authenticated user's past 60 conversation threads for the left sidebar.	Calls `agentApi.listConversations` `conversations` state
`Frontend/src/pages/employee/ChatAssistant.jsx`	`handleSelectConversation(convId)`	Fetches all messages for a specific conversation thread and loads them into the active chat feed.	Calls `agentApi.getConversation(c` sets `messages`
`Frontend/src/pages/employee/ChatAssistant.jsx`	`handleNewChat()`	Resets `activeConversationId = null` and displays the welcome message card.	Focuses `inputRef.current`
`Frontend/src/pages/employee/ChatAssistant.jsx`	`handleDeleteConversation(e, convId)`	Deletes conversation thread from database and removes it from the sidebar list.	Calls `agentApi.deleteConversatio`
`Frontend/src/pages/employee/ChatAssistant.jsx`	`handleSaveTitle(convId)`	Saves user's custom renamed title for a conversation thread.	Calls `agentApi.updateConversatio` `title })`
`Frontend/src/pages/employee/ChatAssistant.jsx`	`handleSend(questionToSend)`	Validates input, checks guest limit (2 free chats), adds user message to state, and sends query with full conversation history.	Calls `agentApi.queryAgent()` , upd opens `<CitationDrawer />` if click
`Frontend/src/api/agent.api.js`	`queryAgent(q, convId, history, widgetId)`	Formats HTTP POST payload containing inquiry text, conversation ID, history array, and optional widgetSessionId.	Calls `apiClient.post('/agent/que`
`Frontend/src/api/agent.api.js`	`getWidgetSession(widgetSessionId)`	Dispatches GET request to restore widget messages by session ID.	Calls `apiClient.get('/agent/widget/s`
`Frontend/src/api/agent.api.js`	`clearWidgetSession(widgetSessionId)`	Dispatches DELETE request to reset widget session in MongoDB.	Calls `apiClient.delete('/agent/widge`
`Frontend/src/api/client.js`	`apiClient request interceptor`	Reads `localStorage.getItem('uk_finance_token')` and attaches `Authorization: Bearer <token>` if present.	Passes config to Axios transport
`Frontend/src/api/client.js`	`apiClient response interceptor`	Automatically unwraps root Axios payload: `(response) ⇒ response.data` .	Returns unwrapped JSON data to cal
`Backend/src/controllers/agent.controller.js`	`askAgent(req, res)`	Validates question, matches or creates MongoDB `Conversation` (by user ID or widgetSessionId), calls RAG service, saves messages.	Calls `processEmployeeQuery()` , th `conversation.save()` , returns `ApiResponse.success()`
`Backend/src/controllers/agent.controller.js`	`getWidgetSession(req, res)`	Finds active widget conversation thread by `widgetSessionId` in MongoDB.	Returns `ApiResponse.success({ c` `messages })`
`Backend/src/controllers/agent.controller.js`	`clearWidgetSession(req, res)`	Deletes widget conversation documents associated with `widgetSessionId` .	Calls `Conversation.deleteMany({` `})`
`Backend/src/services/ragAgent.service.js`	`processEmployeeQuery(question, user, options)`	Master RAG orchestrator: handles greetings, resolves pronouns, checks 4-tier cache, retrieves Qdrant vectors, and synthesizes Groq LLM answer.	Calls `isConversationalFollowUp()` `promptCache.get()` , `searchSimil` `groq.chat.completions()`
`Backend/src/services/ragAgent.service.js`	`isConversationalFollowUp(q, history)`	Regex detector for pronouns (*"they"*, *"them"*, *"their"*, *"it"*, *"that period"*).	Returns boolean to `processEmploye`
`Backend/src/services/ragAgent.service.js`	`buildContextualSearchQuery(q, history)`	Extracts FCA sourcebook codes (`DISP` , `PROD` , `SUP`) from recent user/assistant turns and prepends to query.	Returns rewritten search query string
`Backend/src/services/ragAgent.service.js`	`formatConciseWidgetAnswer(text, askedTable)`	Converts Markdown tables to bullet points when `askedTable` is false; strips trailing disclaimer boilerplate.	Returns concise Markdown string
`Backend/src/services/promptCache.service.js`	`promptCache.get(key, query, model)`	Checks L1 RAM → L2 MongoDB → L3 Qdrant Semantic Cache (≥ 80% cosine similarity).	Returns cached answer or `null`
`Backend/src/services/promptCache.service.js`	`promptCache.set(key, data, tokens, ...)`	Saves answer, tokens, and embedding vector into L1 RAM, L2 MongoDB, and L3 Qdrant collection.	Returns cached entry
`Backend/src/services/vectorStore.service.js`	`searchSimilarRules(query, limit)`	Generates 384-dim dense embedding vector and performs round-robin multi-sourcebook search across Qdrant chunks.	Calls `getEmbeddingModel()` & `searchRuleVectors()`

File Name & Path	Function Name	What It Does Inside	What It Calls Next
`Backend/src/services/qdrant.service.js`	`searchRuleVectors(queryVector, limit)`	Dispatches vector similarity search payload to Qdrant REST client on port 6333.	Returns top matching points with pay